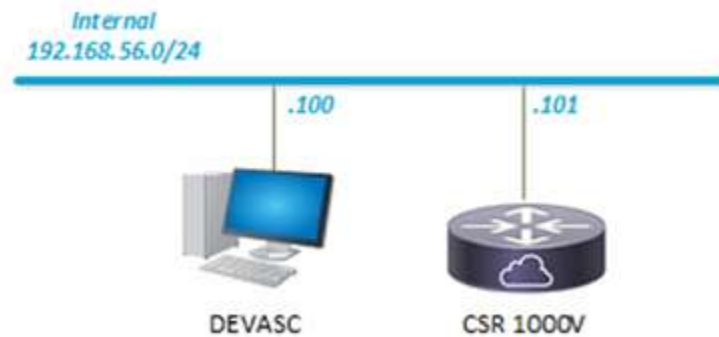


Lab — Configuration Automation using netmiko



Objectives

- 1) Part 1: Checking for netmiko and genie
- 2) Part 2: Using netmiko to create running configuration backup
- 3) Part 3: Using netmiko to apply configuration changes
- 4) Part 4: Using netmiko to verify changes
- 5) Part 5: Using netmiko to rollback failed changes
- 6) Part 6: Testing advanced uses of netmiko and genie

Background / Scenario

In this lab, students will use Python with Netmiko to automate a common network management task that would be conducted on a Cisco IOS device. Students will be able to confirm that the required tools are installed, then connect to the router over SSH to create a backup of the running configuration, apply configuration changes, verify whether those changes were successful, and perform a rollback in the event that the script validation fails. This lab will demonstrate how automation can improve consistency, save time, and ensure proper configurations when deploying in a physical environment.

Required Resources

- Pip3 – pip20.0.2
- Python3 – 3.8.2
- Netmiko – 3.1.1
- Genie – 20.5
- Pyats – 20.5
- DEVASC Lab Environment

- Internet Access
- Visual Code Studio (Recommended)

Instructions

Setup

Please access the DEVASC environment in NetLab+.

- Access Canvas and Github on the DEVASC VM for the required instructional materials.

Part 1: Checking for netmiko and genie

Step 1:

- 1) Start a new linux terminal window
- 2) check for netmiko, genie, and pyats
 - a) pip3 show netmiko
 - b) pip3 show genie
 - c) pip3 show pyats
- 3) Install each if they are not present
 - a) pip3 install netmiko
 - b) pip3 install genie
 - c) pip3 install pyats
- 4) Retry the show commands

Part 2: Using netmiko to create running configuration backup

- 1) Create a folder named “mikolab” on the desktop
- 2) Create new file in mikolab, and rename it to automation.py
- 3) Open automation.py in Visual Code Studio
- 4) Type in the python script below, pay attention to the comments describing what is happening:
 - a) You do not need to type the words that are commented out

Note: If typing is not possible, the document should allow for copy-paste

```
from netmiko import ConnectHandler
```

```

import re
import time

# =====
# ROUTER LOGIN INFORMATION
# =====
# This dictionary stores the login information for the router.
# Netmiko uses these values to open an SSH connection.
device = {
    "device_type": "cisco_ios",
    "host": "192.168.56.101",
    "username": "cisco",
    "password": "cisco123!",
}
# This is the file where we will save the router backup.
backup_file = "router_backup.cfg"

# =====
# PHASE 1 - CONNECT AND BACK UP THE CONFIG
# =====

print("PHASE 1 - CONNECT TO THE ROUTER AND CREATE A BACKUP")
try:
    print("Connecting to the router with Netmiko")
    # ConnectHandler() opens an SSH session to the router.
    # The **device part means:
    # use all the values from the device dictionary as settings.
    connection = ConnectHandler(**device)
    print("Connected.\n")

    print("Requesting the running configuration")
    running_config = connection.send_command("show running-config")
    print("Configuration received.")
    time.sleep(1)

```

```

print(f'Saving backup to {backup_file}')
time.sleep(1)
# This opens a file on the Linux machine and writes
# the running config into it.
with open(backup_file, "w") as f:
    f.write(running_config)
print("Backup complete.\n")
time.sleep(1)

except Exception as e:
    print(f'Error during Phase 1: {e}')
connection.disconnect()

```

Biggest troubleshooting issues:

Bad formatting, like improper indents

Bad repo versions

Not running script from the script directory

Best Troubleshooting Tips:

Run script from visual code studio

Run script with python3 command while terminal is in the script's directory

Make sure package versions are correct

If formatting is incorrect, copy and paste

5) Once you have looked through the code, try and figure out what is happening.

6) Use Visual Code Studio to run the script

a) Does it work? **It should**

7) Use the terminal in VCS to run "nano ./router_backup.cfg"

a) Do you see the running config of the router? **It should be saved in whatever directory the script was run from. Visual Code Studio defaults to home directory**

[Screenshot of router_backup.cfg contents]

Part 2 Summary: The code above uses netmiko and a dictionary containing device information to ssh into the router. Once the ssh has successfully connected, netmiko's send_command function is used to operate the "show running-config" command as an enabled privilege level. The python captures the output from the running-config command and writes it to the router_backup.cfg file detailed at the beginning of the script. The final line connect.disconnect tells the ssh channel to close.

Part 3: Using netmiko to apply configuration changes

- 1) In Visual Code Studio, delete the "connection.disconnect" at the bottom.
- 2) You will keep the existing code from Part 2, and add the following to the bottom:

```
# =====
# PHASE 2 - APPLY CONFIGURATION CHANGES
# =====

print("\n=====")
print("PHASE 2 - APPLY CONFIGURATION CHANGES")
print("=====\n")
time.sleep(1)

try:
    print("We will now change two settings on the router:")
    print("1. The hostname")
    print("2. The GigabitEthernet1 description\n")
    time.sleep(2)

    # Ask the student for a new hostname.
    # In this lab environment, using a space in the hostname
    # should cause the hostname change to fail.
    new_hostname = input("Enter a new hostname: ").strip()

    # Ask the student for a new interface description.
    new_description = input("Enter a new GigabitEthernet1 description: ").strip()
```

```

print("\nBuilding configuration commands...")
time.sleep(1)

# This list stores the commands we want to send.
config_commands = [
    f"hostname {new_hostname}",
    "interface GigabitEthernet1",
    f"description {new_description}"
]

print("Sending the commands to the router...\n")
time.sleep(1)

# send_config_set() is used for configuration commands.
# Netmiko automatically enters config mode,
# sends the commands, and exits config mode.
config_output = connection.send_config_set(config_commands)

print("Router response:\n")
print(config_output)
time.sleep(1)

print("Configuration changes sent.\n")
time.sleep(1)

except Exception as e:
    print(f"Error during Phase 2: {e}")
connection.disconnect()

```

- 3) Take a look at the added functions and the commented lines describing what is happening.

- 4) Save the python file and run it like previously, the command prompt will ask questions.
 - a) Please, do not add special characters or spaces to any of the configurations.

- 5) Go over to the router command line interface, and show running-config

- a) What has changed? The hostname should be different, so should be the interface description for GigabitEthernet1
- b) Is whats on the router different than what is in the backup file? If the script worked, yes

[Screenshot of the router running config]

[Screenshot of the terminal after the code finishes]

Part 3 Summary: In this section you added code onto what was presented in part 2, the new code allows you to make modifications to the router's configurations after a backup is created. By using input, we can ask the user to specify what they want to be configured, which is placed into a list and then the list is run through the send_command function. This allows commands to be run on any amount of routers, switches, or cisco devices with a sweeping configuration, all that needs to be done is alter the device information from part 2.

Part 4: Using netmiko to verify changes

- 1) In Visual Code Studio, delete the “connection.disconnect” at the bottom.
- 2) You will keep the existing code from Part 2 and 3, and add the following to the bottom:

```
# =====  
  
# PHASE 3 - VALIDATE THE CHANGES  
  
# =====  
  
print("\n=====")  
print("PHASE 3 - VALIDATE THE CHANGES")  
print("=====\\n")  
time.sleep(1)  
  
try:  
    print("Checking the hostname in the running config...")  
    time.sleep(1)
```

```

# This asks the router to show only the hostname line.
hostname_output = connection.send_command("show running-config | include ^hostname")

# This checks whether the hostname we wanted is actually present.
# The result will be either True or False.
hostname_ok = f"hostname {new_hostname}" in hostname_output

print("Checking the GigabitEthernet1 description...")
time.sleep(1)

# This asks the router to show the config for GigabitEthernet1.
desc_output = connection.send_command("show running-config interface GigabitEthernet1")

# This checks whether the description we wanted is present.
description_ok = f"description {new_description}" in desc_output

print("\nValidation Results:")
print("-----")
print(f'Hostname: {'Pass' if hostname_ok else 'Fail'}')
print(f'Interface Description: {'Pass' if description_ok else 'Fail'}\n')
time.sleep(2)

except Exception as e:
    print(f'Error during Phase 3: {e}')
connection.disconnect()

```

- 3) Take a look at the added functions and the commented lines describing what is happening.
- 4) Save the python file and run it like previously, the command prompt will ask questions.
 - c) Please run it without using special characters in the prompts

d) Did you configurations pass verification? If they used the right formatting, then it should have worked. If it does not work, they may need to try again with a different answer to the prompts.

6) Now re-run the script, except use a space in hostname, example "Cisco Router"

a) Did it verify successfully? Why? It should not succeed, CSR 1000v does not support space characters in the hostname

[Screenshot of successful verifications]

[Screenshot of failed verification]

Part 4 Summary: [Placeholder]

Part 5: Using netmiko to rollback failed changes

1) In Visual Code Studio, delete the “connection.disconnect” at the bottom.

2) You will keep the existing code from Part 2, 3. and 4, and add the following to the bottom:

```
# =====  
  
# PHASE 5 - ROLLBACK IF VALIDATION FAILED  
  
# =====  
  
print("\n=====")  
print("PHASE 5 - ROLLBACK IF VALIDATION FAILED")  
print("=====\\n")  
time.sleep(1)  
  
try:  
    # If both checks passed, leave the config as-is.  
    if hostname_ok and description_ok:  
        print("Both changes passed validation.")  
        print("No rollback is needed.\\n")  
        time.sleep(1)  
  
    else:  
        print("Validation failed.")
```

```
print("Even if one change worked, both changes will be rolled back.")
time.sleep(2)

rollback_commands = []

# Put the original hostname back.
if original_hostname:
    rollback_commands.append(f"hostname {original_hostname}")

# Enter the interface section.
rollback_commands.append("interface GigabitEthernet1")

# If there was an old description, restore it.
# If there was no old description, remove the current one.
if original_description:
    rollback_commands.append(f"description {original_description}")
else:
    rollback_commands.append("no description")

print("\nSending rollback commands to the router...\n")
time.sleep(1)

rollback_output = connection.send_config_set(rollback_commands)

print("Router response:\n")
print(rollback_output)
time.sleep(1)

print("Checking rollback results...\n")
time.sleep(1)
```

```

# Re-check the hostname after rollback.

hostname_output = connection.send_command("show running-config | include ^hostname")

if original_hostname:
    hostname_rollback_ok = f"hostname {original_hostname}" in hostname_output
else:
    hostname_rollback_ok = False

# Re-check the interface description after rollback.

desc_output = connection.send_command("show running-config interface GigabitEthernet1")

if original_description:
    description_rollback_ok = f"description {original_description}" in desc_output
else:
    description_rollback_ok = "description " not in desc_output

print("Rollback Results:")
print("-----")
print(f"Hostname Rollback: {'Passed' if hostname_rollback_ok else 'Failed'}")
print(f"Interface Description Rollback: {'Passed' if description_rollback_ok else 'Failed'}\n")
time.sleep(2)

except Exception as e:
    print(f"Error during Phase 5: {e}")
connection.disconnect()

```

- 3) Take a look at the added functions and the commented lines describing what is happening.
- 4) Save the python file and run it like previously, the command prompt will ask questions.

- b) Please run it without using special characters in the prompts and please use something different than last steps
 - c) What happens with the code? The script should print out new lines, stating verification succeeded and there is no need for rollback
- 7) Now re-run it with the space in the hostname, example "Cisco Router" and please use something different than last steps for description
- a) How does it behave differently from part 4? Once it gets past the verification step, the script should immediately run a rollback which will return the settings back to what they were from the previous script's execution
- 8) Check the router, is the hostname what you tried to configure it as? It should not be the same, since the previous step had them use an invalid hostname
- a) What about the interface description? It should not be since the script rolls everything back if only 1 point of failure is detected
 - b) Why is the interface description not what you configured it as despite only the hostname failing? The script rolls everything back if only 1 point of failure is detected

[Screenshot of the rollback success]

Part 5 Summary: [Placeholder]